Universidad del Rosario | Escuela de Ciencias e Ingeniería | MACC

Mecánicas de C++

Punteros, memoria y el patrón iterador

Objetivos

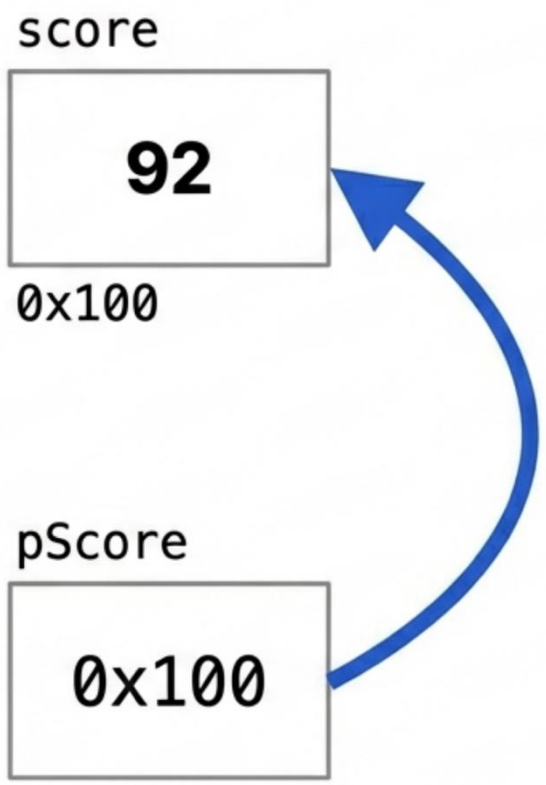
- Repasar la creación de arreglos en memoria dinámica (**new**) y su liberación (**delete[ ]**).
- Comprender la aritmética de punteros como mecanismo de desplazamiento en memoria.
- Descubrir que un puntero es un iterador (específicamente, un iterador de acceso aleatorio).
- Aplicar el patrón **begin / end** usando punteros puros para recorrer un arreglo sin usar una variable entera i.

El Puntero: Una Variable para Direcciones

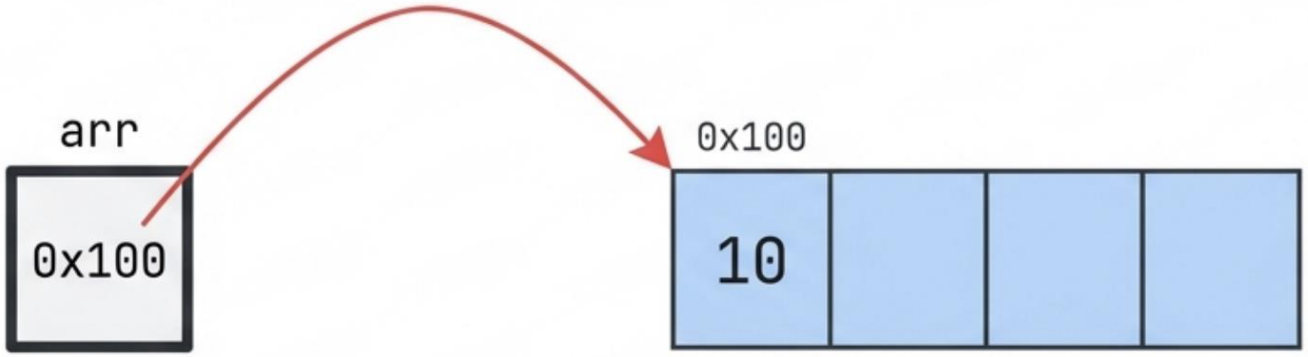
Un puntero no es magia; es una variable que almacena una dirección de memoria.

- **Declaración:** **type* ptr**
- **Asignación:** **ptr = &var**
- **Desreferencia:** ***ptr** (accede al valor apuntado)

```
int score = 92;
int* pScore = &score; // Acceso indirecto
cout << *pScore; // Imprime: 92
```

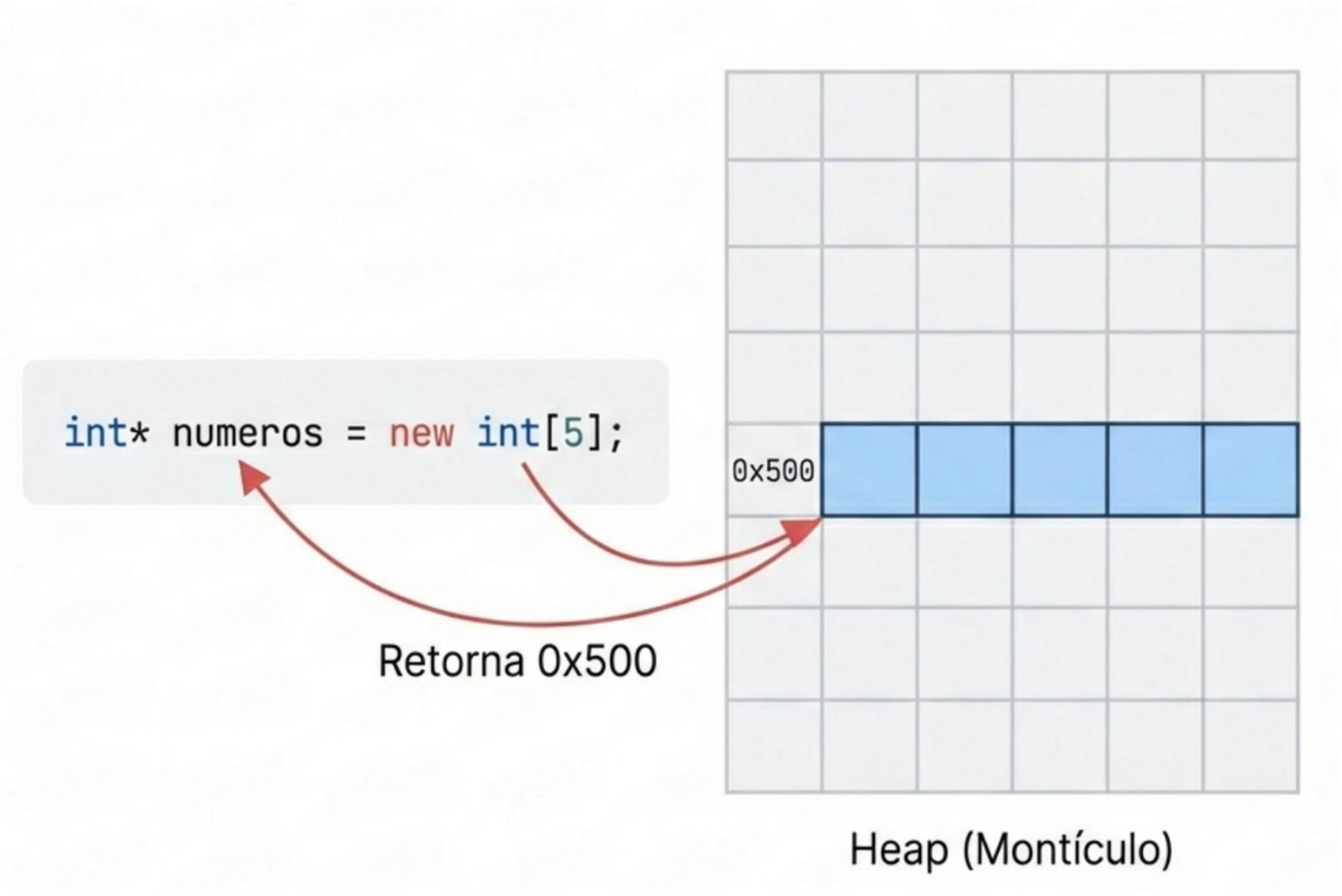


La variable vs los datos



En C++, la variable **arr** no contiene los datos. Es solo una variable pequeña que guarda la dirección de memoria donde comienzan los datos.

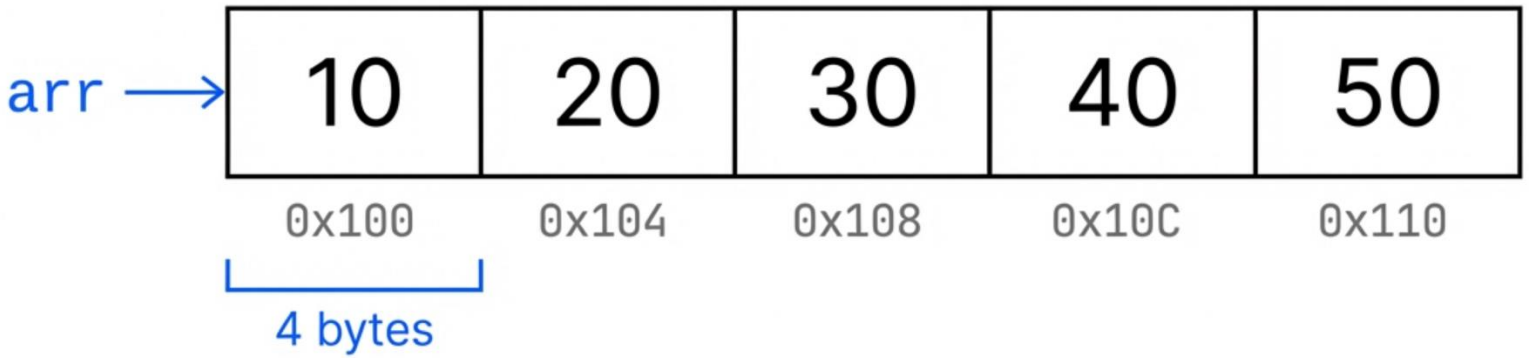
Reserva de la memoria



Proceso:

1. **Solicitud:** `new` pide 20 bytes (5 enteros x 4 bytes).
2. **Búsqueda:** El sistema encuentra espacio libre contiguo en el Heap.
3. **Asignación:** Devuelve la dirección del primer byte a la variable.

El arreglo es una dirección

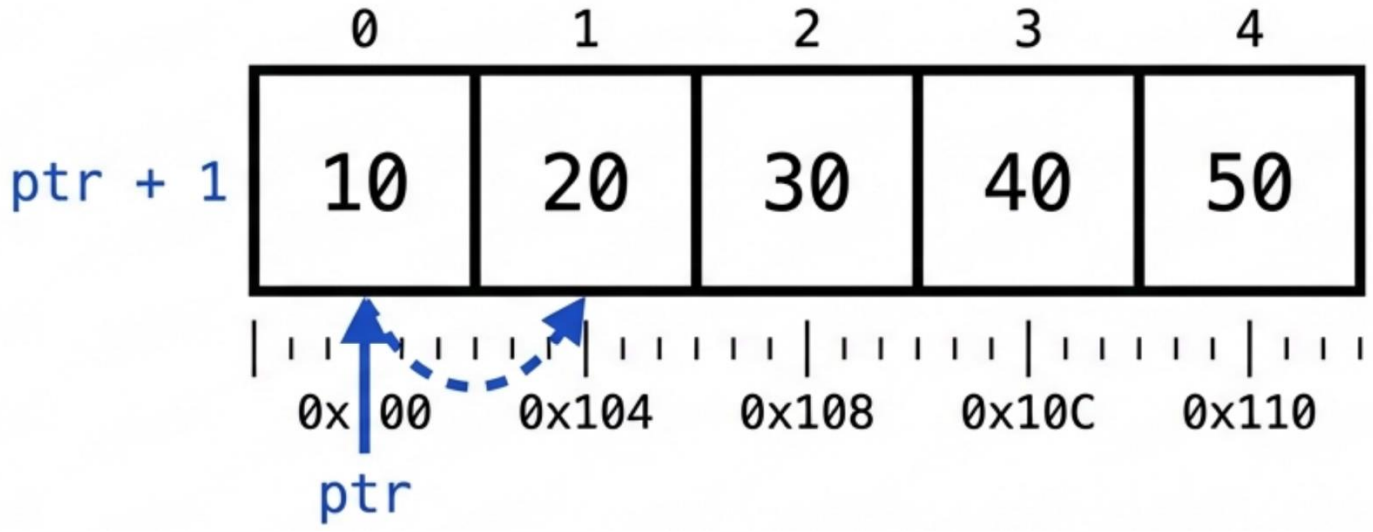


En la memoria RAM, un arreglo no es una lista numerada. Es un bloque contiguo de bytes. La variable **arr** es en realidad un puntero constante al primer elemento.

```
arr == &arr[0]
```

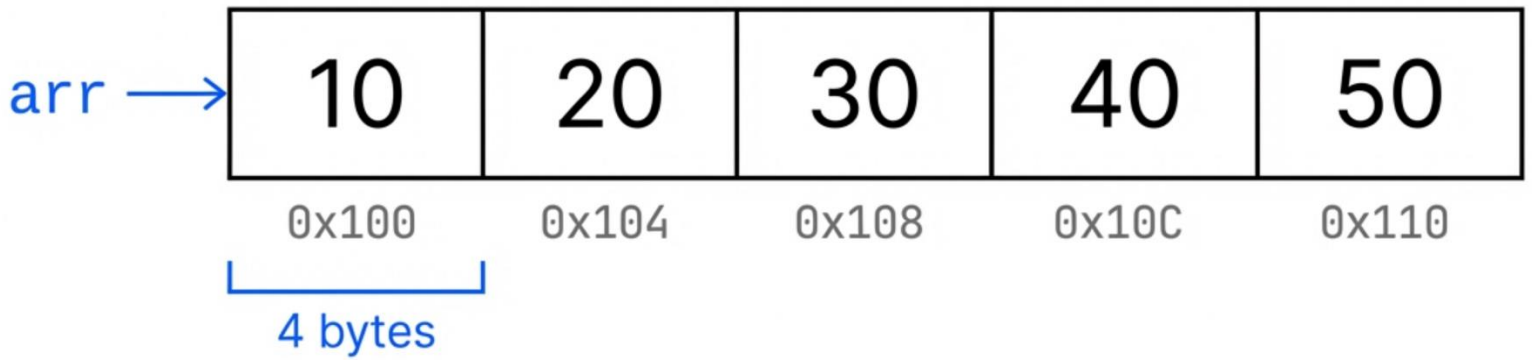
Aritmética de Punteros y Arrays

En C++, el nombre de un array es un puntero al primer elemento. La aritmética de punteros es inteligente: **ptr + 1** no avanza ***1 byte***, sino **sizeof(type)** bytes.




```
int arr[5] = {10, 20, 30, 40, 50};
int* ptr = arr;
cout << *(ptr + 1); // Imprime 20
```

La ilusión de la continuidad



Lo que escribimos

`arr[i]`

Azúcar sintáctica
(Syntactic Sugar)

Lo que sucede

`*(arr + i)`

Dirección base
(Inicio)

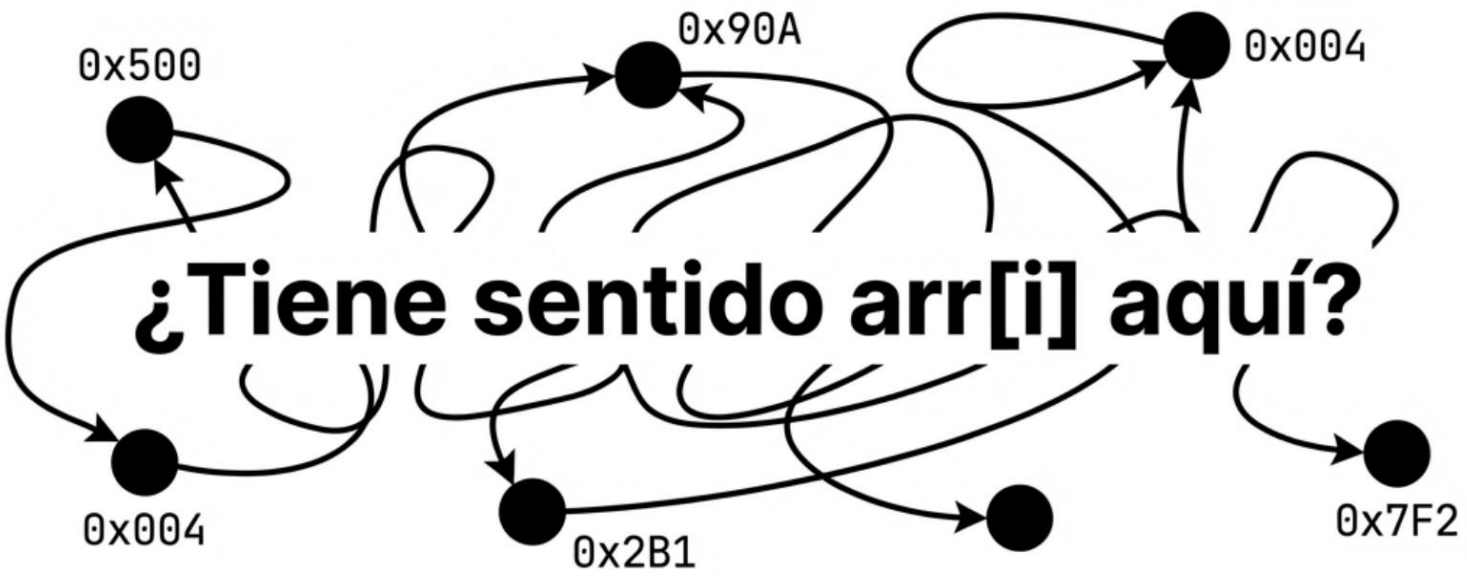
Caminar i pasos

Mirar dentro (Desreferenciar)

El índice **arr[i]** funciona porque es pura aritmética: ***inicio + (i * tamaño)***.

Esta lógica asume que los datos viven '***puerta con puerta***' en la memoria física. Dependemos de la ubicación física, no de la relación lógica.

¿Qué pasa cuando rompemos la fila?



Estructuras como Listas Enlazadas o Árboles dispersan los datos en la memoria. La aritmética simple (**i++**) nos llevaría a basura en la memoria, no al siguiente dato.

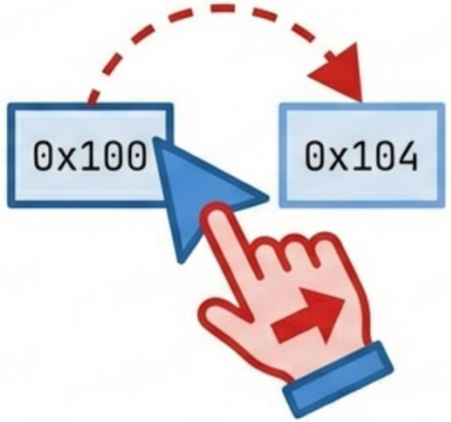
Mirar vs Mover

Desreferencia `*p`

Acceder

Preguntar: "¿Qué valor hay aquí?"

Incremento `p++`

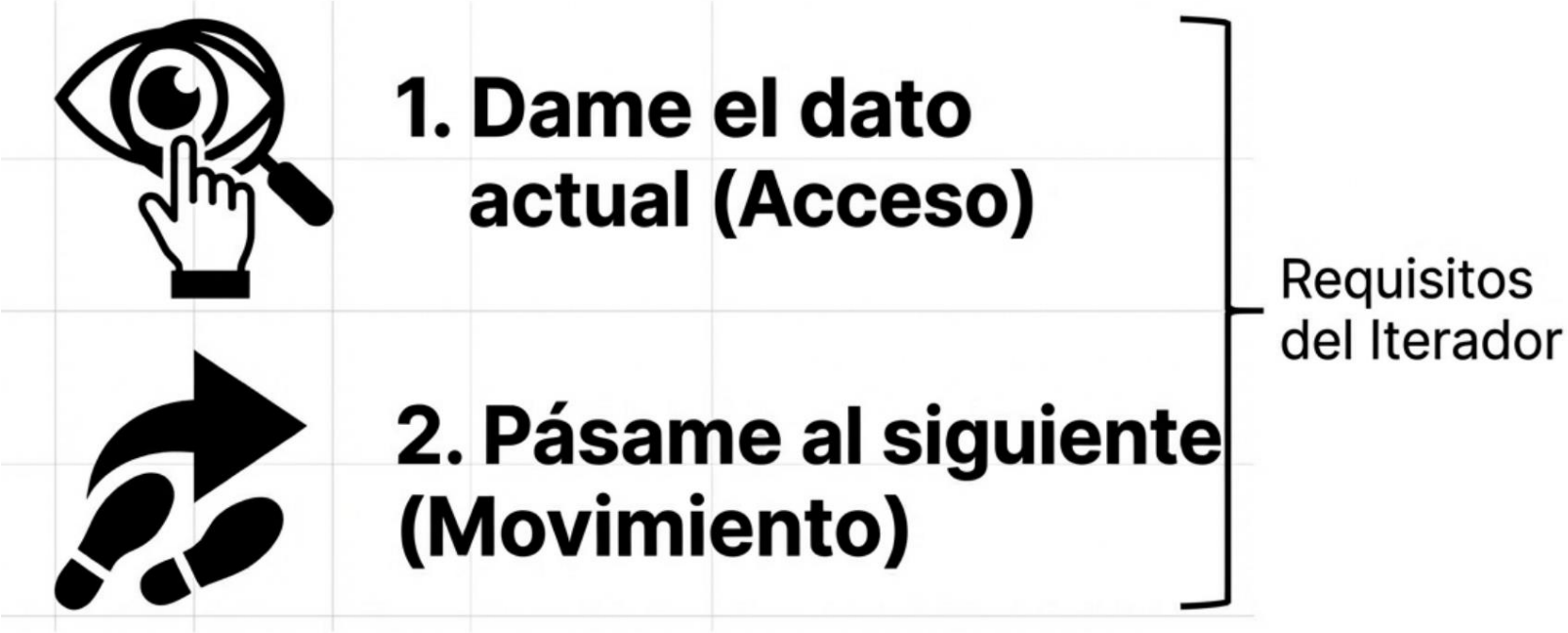


Desplazar

Modificar: "Apunta a la siguiente casilla".

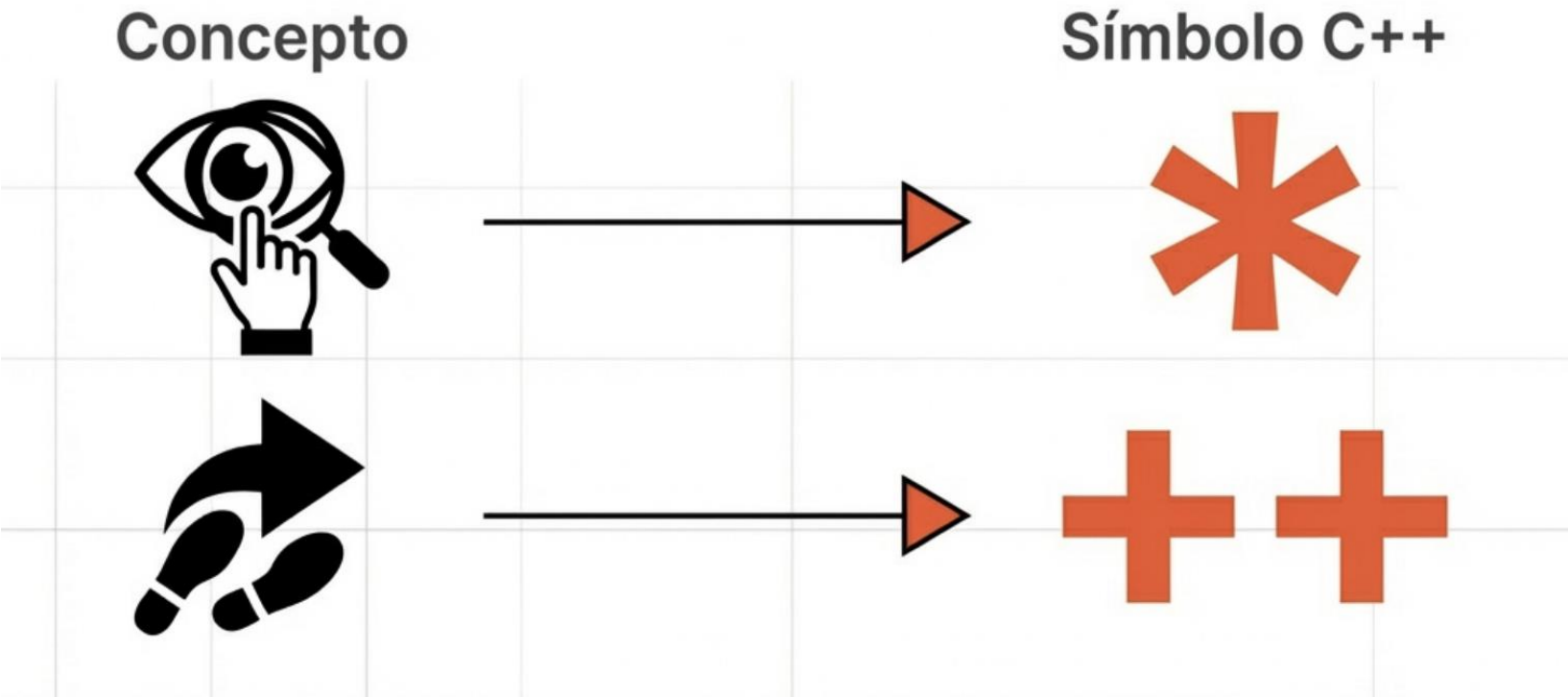
Necesitamos una abstracción

Necesitamos un “viajero universal” que pueda recorrer cualquier estructura sin conocer su mapa de memoria. Este “viajero universal” le vamos a dar el nombre de ***iterador***



Necesitamos una abstracción

Necesitamos un “viajero universal” que pueda recorrer cualquier estructura sin conocer su mapa de memoria. Este “viajero universal” le vamos a dar el nombre de ***iterador***



Iteradores

ANTES (Índice)

`arr[i]`

Acceso directo por aritmética.

`i++`

Suma de bytes a la dirección base.

AHORA (Iterador)

`*iter`

Acceso al objeto actual (Derreferencia).

`iter++`

Lógica interna para encontrar el siguiente nodo.

Para ti, la sintaxis es simple. Para la máquina, 'iter++' sabe mágicamente si debe sumar 4 bytes o seguir un puntero en el Heap.

El concepto de rango

[begin, end)

- Dejamos de pensar en índices (0 a 4).
- Pensamos en límites físicos de memoria.
- **begin**: Dónde empiezan los datos.
- **end**: Dónde termina el espacio válido.

Configuración de iteradores

```
const int N = 5;
// Inicialización del array
int* numeros = new int[N]{10, 20, 30, 40, 50};

// 1. Puntero al inicio
int* begin = numeros;

// 2. Puntero al final (Centinela)
int* end = numeros + N;
```

Apunta al índice 0

Aritmética: Salta 5 posiciones hasta el vacío

Bucle de punteros

Estilo Clásico (Índices)

```
for (int i = 0; i < N; i++) {
    ...
}
```

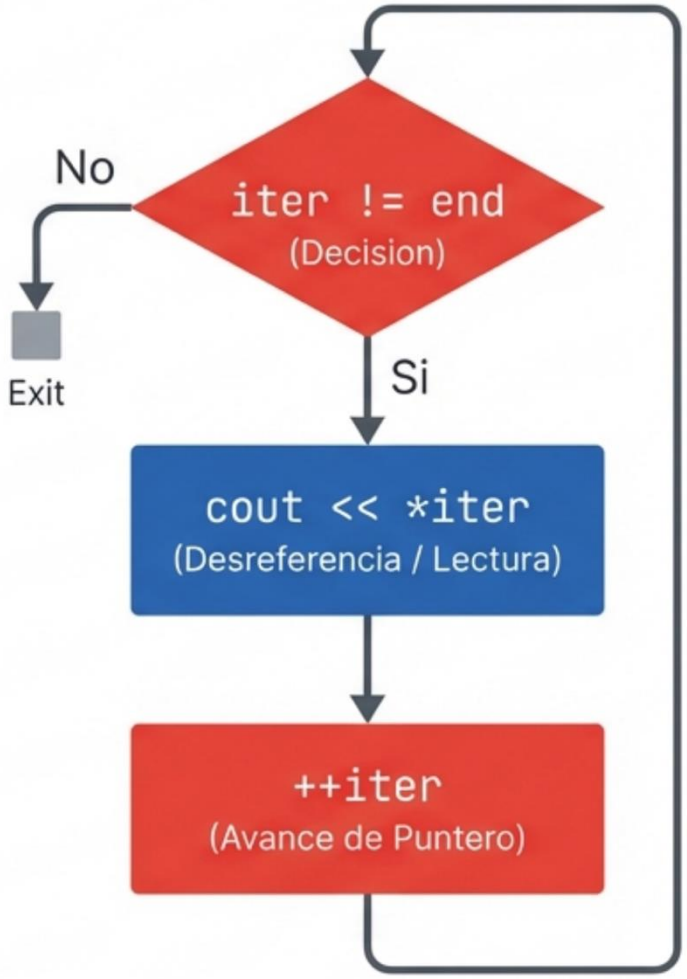
Estilo Iterador (Punteros)

```
for (int* iter = begin; iter != end; ++iter) {
    cout << *iter << " ";
}
```

Seguridad:
¿Chocamos con el centinela?

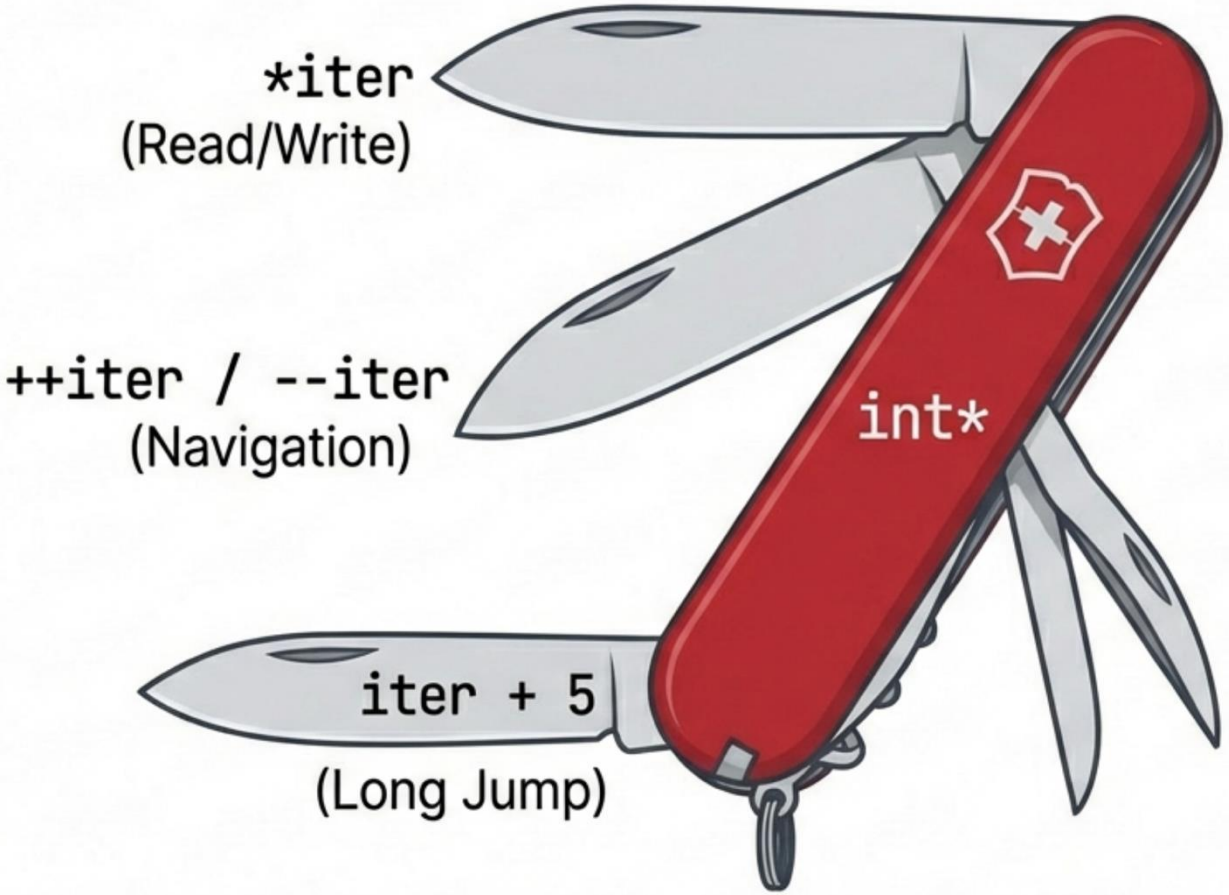
Avance físico

Anatomía del ciclo




```
for (int* iter = begin; iter != end; ++iter)
```

¿Para que hacer todo esto?



Este puntero simple es el iterador más poderoso de C++.

Es la base de `std::vector` y `std::array`.

Iterando en C++11

```
std::vector<int> v = {1, 2, 3};

// El bucle universal
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it;
}
```

Apunta al primer elemento

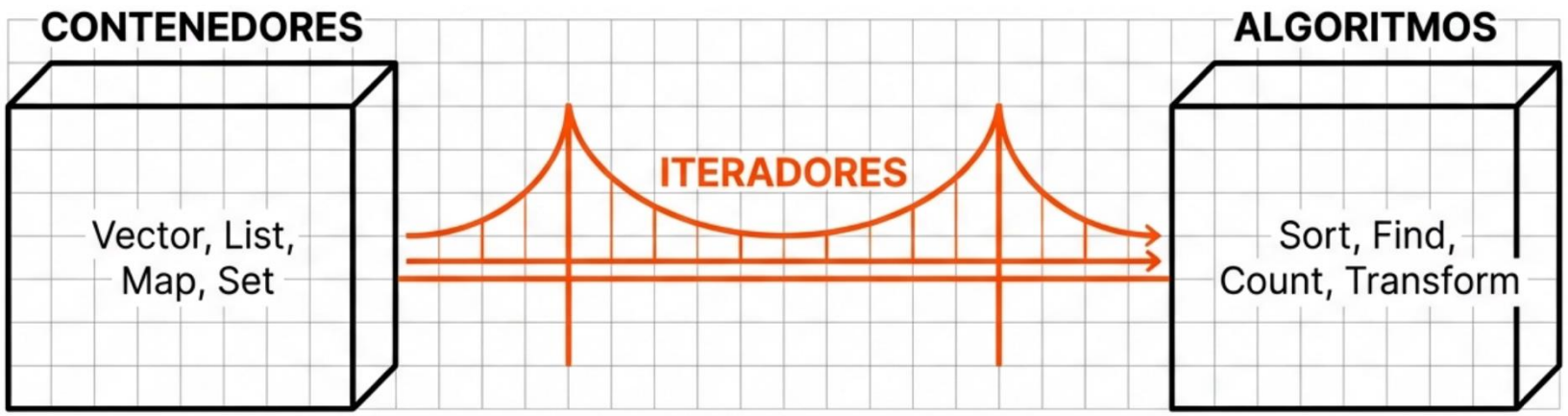
Apunta PASADO el último elemento (el muro de contención)

El iterador encapsula el movimiento

Los iteradores son la llave al reino STL

Standard Template Library (Biblioteca de Plantillas Estándar)

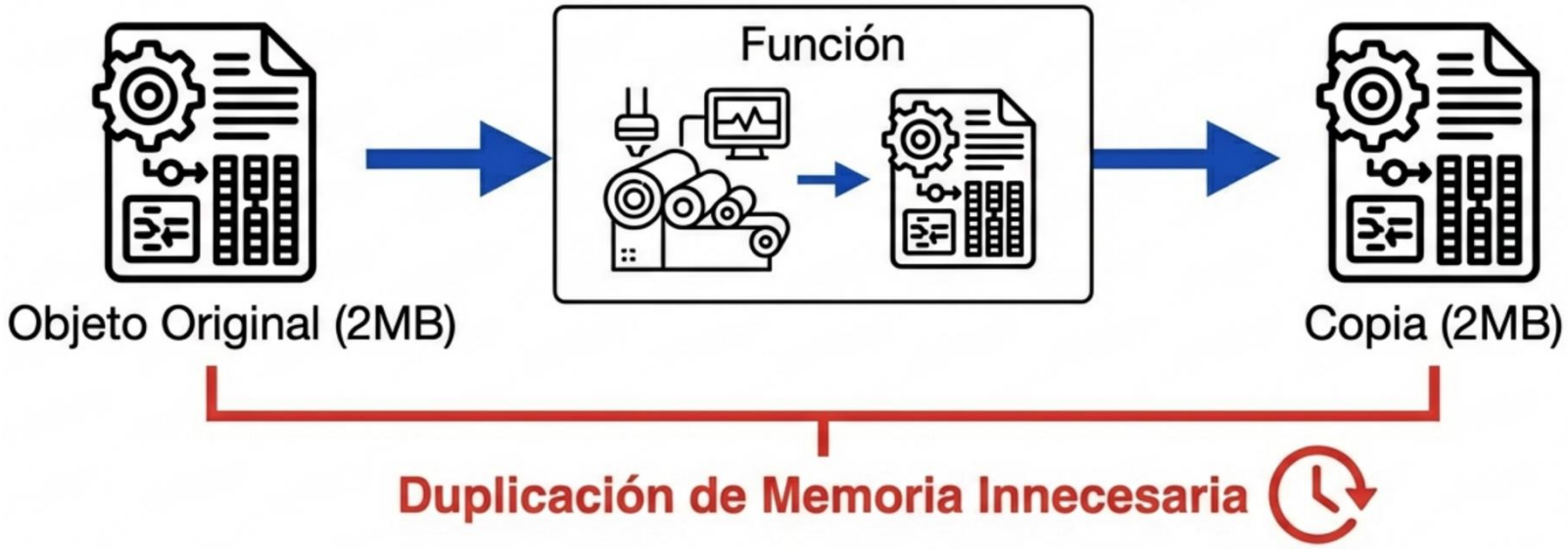
01_iter.cpp



Entender los iteradores desbloquea toda la biblioteca estándar. Los algoritmos no saben de contenedores; solo saben de iteradores.

Referencias

Evitando el costo del paso por valor



Referencias

Una referencia (**&**) actúa como un alias. No se crea una nueva variable; simplemente se le asigna un segundo nombre a la ubicación de memoria existente. El acceso es inmediato y sin costo de copia.

```
void procesar(int& ref) {
  ref++; // Modifica el original
}

int main() {
  int x = 10;
  procesar(x); // x ahora es 11
}
```

Comparativa: Punteros vs. Referencias

Referencias (int&)	Punteros (int*)
• No pueden ser nulas (siempre inicializadas). • Vinculación permanente (no reasignables). • Sintaxis limpia (uso directo).	• Pueden ser nullptr. • Reasignables (pueden cambiar de dirección). • Sintaxis explícita (* y ->).

"Una referencia es una relación constante que no puede romperse."

Eficiencia y Seguridad: **const Ref**

La regla de oro en C++ moderno para pasar objetos complejos:

const Type&

Esto combina la eficiencia de la referencia (sin copia) con la seguridad del paso por valor (inmutabilidad).

```
void imprimir(const string& texto) {
  cout << texto; // OK: Lectura eficiente
  // texto = "Hola"; // ERROR: Solo lectura
}
```

Función Swap

Estilo C (Punteros)

```
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
// Uso: swap(&x, &y);
```

Estilo C++ (Referencias)

```
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}
// Uso: swap(x, y);
```

✅ Más limpio. Menos propenso a errores.

02_ref_swap.cpp

Resumen

1. El Puntero es el Iterador Universal

- No necesitamos índices [i].
- Usamos **Direcciones de Memoria** para navegar.
- **Aritmética:** ptr++ salta al siguiente elemento físico (depende del sizeof).

2. El Patrón [Begin, End)

- **begin:** Donde empieza la fiesta.
- **end:** El "Centinela" (Zona Prohibida). Apunta **uno después** del último dato.
- *Regla:* El bucle corre mientras it != end.

3. Eficiencia: Paso por Referencia (&)

- **Por Valor:** Sacamos fotocopia (Lento, no modifica original).
- **Por Referencia:** Entregamos la llave de la casa (Rápido, modifica original).
- Vital para ordenar, filtrar o transformar datos "in-place".